



---

# ИНТЕРАКТИВНЫЙ КОНТЕЙНЕР ДЛЯ СОРТИРОВКИ БАТАРЕЕК

---





## РЕФЕРАТ

### **Объем пояснительной записки:**

- Количество страниц - 34
- Количество иллюстраций - 6
- Количество таблиц - 2
- Количество использованных источников - 6
- Количество приложений - 1

**Ключевые слова:** «умный» контейнер, экологическое воспитание, сортировка отходов, машинное зрение, нейросетевое распознавание, Arduino Uno, Raspberry Pi, 3D-печать, лазерная резка.

**Объект разработки:** интерактивный контейнер для сбора батареек.

**Цель:** спроектировать и изготовить модель «умного» контейнера.

**Методы:** сравнительный анализ аналогов, проектирование в КОМПАС-3D, изготовление деталей на 3D-принтере и лазерном станке, программирование на Python, C++.

**Результаты и новизна:** создан прототип. Новизна — в интеграции нейросети в общедоступное эко-устройство, что обеспечивает мотивацию и аналитику собранных отходов.

**Область применения:** школы, офисы, торговые центры.

**Рекомендации по внедрению:** пробное тестирование в школах. Для серийного производства — удешевление конструкции.

**Значимость:** снижение экологического ущерба, повышение вовлечённости в сортировку отходов.

**Перспективы развития:** Создание облачной платформы для сбора данных со всех контейнеров, обучение нейросети на новых типах отходов и адаптация для их сбора.

# СОДЕРЖАНИЕ

|                                                                                                           |    |
|-----------------------------------------------------------------------------------------------------------|----|
| ВВЕДЕНИЕ.....                                                                                             | 5  |
| Актуальность темы проекта и проблема, решаемая в проекте .....                                            | 5  |
| Цель проекта.....                                                                                         | 6  |
| Задачи .....                                                                                              | 6  |
| ОСНОВНАЯ ЧАСТЬ.....                                                                                       | 7  |
| 1        ПОИСКОВО-ИССЛЕДОВАТЕЛЬСКИЙ ЭТАП.....                                                             | 7  |
| 1.1    Результаты маркетингового исследования .....                                                       | 7  |
| 1.2    Характеристики проектируемого объекта труда,<br>значимые для потенциальной целевой аудитории ..... | 7  |
| 1.3    Анализ прототипов и аналогов.....                                                                  | 7  |
| 1.4    Требования к проекту.....                                                                          | 8  |
| 2        КОНСТРУКТОРСКО-ТЕХНОЛОГИЧЕСКИЙ ЭТАП .....                                                        | 9  |
| 2.1    Методы проектирования при разработке конструкции.....                                              | 9  |
| 2.2    Методы дизайнерской работы.....                                                                    | 9  |
| 2.3    Выбор материалов .....                                                                             | 9  |
| 2.4    Использование АРИЗ .....                                                                           | 9  |
| 2.5    Технологические карты.....                                                                         | 10 |
| 2.6    Выбор инструментов, оборудования, приспособлений и<br>технологии изготовления .....                | 12 |
| 2.7    Описание принципа работы.....                                                                      | 13 |
| 2.8    Электрическая схема .....                                                                          | 14 |
| 3        ЗАКЛЮЧИТЕЛЬНЫЙ ЭТАП .....                                                                        | 15 |
| 3.1    Экономическая оценка: .....                                                                        | 15 |
| 3.2    Экологическая оценка .....                                                                         | 15 |
| 3.3    Описание характеристик проектного изделия.....                                                     | 15 |

|     |                                                                                          |    |
|-----|------------------------------------------------------------------------------------------|----|
| 3.4 | Уникальность проектного решения .....                                                    | 16 |
| 3.5 | Перспектива тиражируемости .....                                                         | 16 |
| 3.6 | Перспектива масштабируемости .....                                                       | 16 |
| 4   | ЗАКЛЮЧЕНИЕ .....                                                                         | 17 |
| 4.1 | Краткие выводы по результатам проекта .....                                              | 17 |
| 4.2 | Оценка полноты решений поставленных задач .....                                          | 18 |
| 4.3 | Рекомендации и исходные данные по конкретному<br>использованию результатов проекта ..... | 18 |
| 4.4 | Результаты технико-экономической оценки внедрения<br>проекта .....                       | 18 |
| 4.5 | Результаты оценки научно-технического уровня<br>выполненного проекта .....               | 19 |
| 5   | СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ .....                                                   | 20 |
|     | ПРИЛОЖЕНИЕ .....                                                                         | 21 |

## **ВВЕДЕНИЕ**

Современный технологический прогресс и рост потребления электронных устройств привели к увеличению объема опасных отходов, таких как батарейки. Согласно данным экологов, одна пальчиковая батарейка, попадая на свалку, способна отравить до 20 квадратных метров почвы и сотни литров грунтовых вод, нанося серьезный ущерб экосистеме и здоровью человека. Однако, даже при высокой осведомленности населения об этой угрозе, регулярно сдают батарейки на переработку менее 10% граждан.

Главная проблема в том, что обычные контейнеры не мотивируют. Они лишь накапливают мусор, не давая человеку ощущения своего вклада в этот процесс и удобства пользования контейнером.

Данный проект направлен на разработку и создание функционального прототипа интерактивного контейнера для сбора и первичной сортировки батареек. Его принципиальное отличие — интеграция технологии машинного зрения на основе нейросети, что позволяет устройству не только принять отходы, но и в реальном времени распознать их тип, обеспечить пользователя мгновенной визуальной обратной связью, вести учет и наглядно демонстрировать личный вклад в сохранение окружающей среды.

### **Актуальность темы проекта и проблема, решаемая в проекте**

Актуальность проекта связана с серьёзной экологической проблемой утилизации батареек. Они содержат токсичные металлы (ртуть, свинец, кадмий), и всего одна батарейка, попав на свалку, может загрязнить 20 м<sup>2</sup> почвы и 400 л воды, нанося вред природе и здоровью людей. При этом батарейки — ценное сырьё, которое можно перерабатывать, но без сортировки ресурсы теряются.

Основная проблема — низкая вовлечённость людей. Многие знают о вреде, но не сортируют батарейки из-за отсутствия мотивации и удобства: обычные контейнеры неинформативны и не дают ощущения личного вклада. Поэтому проект актуален не только с экологической, но и с социальной точки

зрения — он предлагает технологическое решение для преодоления безразличия через интерактивность и обратную связь.

### **Цель проекта**

Разработка конструкции, изготовление и программирование функционального прототипа интерактивного контейнера для сбора, сортировки и временного хранения использованных батареек.

### **Задачи**

- 1 Провести анализ существующих аналогов и сформулировать технические требования к устройству.
- 2 Разработать функциональную и конструкторскую схемы устройства.
- 3 Спроектировать 3D-модели деталей корпуса и механизмов.
- 4 Разработать и собрать электрическую схему устройства.
- 5 Разработать архитектуру и обучить сверточную нейросеть для классификации изображений батареек по основным типам (пальчиковые (AA), крона и не батарейки). Создать и разметить собственный датасет.
- 6 Реализовать программную связку: микрокомпьютер Raspberry Pi (обработка изображения, работа нейросети) и микроконтроллер Arduino Uno (управление исполнительными механизмами и интерфейсом).
- 7 Написать управляющие программы для обеих платформ.
- 8 Изготовить детали, произвести сборку и испытания прототипа.

# ОСНОВНАЯ ЧАСТЬ

## 1 ПОИСКОВО-ИССЛЕДОВАТЕЛЬСКИЙ ЭТАП

### 1.1 Результаты маркетингового исследования

На основе исследования Ассоциации содействия экономике замкнутого цикла «Ресурс» и компании ИНСОМАР и социального опроса группы «М.Видео-Эльдорадо» сделан вывод о высоком уровне осведомленности населения о вреде батареек и положительным отношением к сортировке отходов (более 80%), но довольно низком проценте их правильной утилизации (менее 25%) [1-3]. Это доказывает, что нужен не просто обычный контейнер, а решение, которое вовлекает и мотивирует людей действовать. Спрос существует именно на интерактивный и понятный продукт.

### 1.2 Характеристики проектируемого объекта труда, значимые для потенциальной целевой аудитории

Для потенциальных пользователей (школьников, студентов, сотрудников офисов) важны: интерактивность (обратная связь), простота использования, наглядность пользы и эстетичный дизайн, вписывающийся в интерьер. Эти характеристики легли в основу технического задания.

### 1.3 Анализ прототипов и аналогов

В таблице 1 приведены примеры двух основных типов аналогов [4-5].

*Таблица 1 Анализ аналогов*

| Аналоги                                                                                                 | Достоинства                                                                                  | Недостатки                                                             |
|---------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------|------------------------------------------------------------------------|
| Экообменник ITES<br> | Мотивирует людей к сдаче батареек путем выдачи бонусов магазина за каждую сданную батарейку. | Не сортирует батарейки по типам и всё попадает в один общий контейнер. |



|                                                                                                                        |                                                     |                                                                                                                            |
|------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------|
| <p>Контейнеры для сбора батареек</p>  | <p>Простота и дешевизна. Яркий красивый дизайн.</p> | <p>Они совсем не вовлекают человека и не сортируют батарейки по типам. Это просто коробка, в которую кидают батарейки.</p> |
|------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------|

Существующие решения либо слишком примитивны и не меняют поведение людей, либо сделаны для удобства не пользователя, а обслуживающего персонала. Поэтому мой проект — интерактивный контейнер, который общается с человеком — является актуальным и нужным. Я фокусируюсь на том, чтобы пользователю было приятно и интересно сдавать батарейки. Также существующие аналоги не обладают функцией автоматической идентификации типа батареек, поэтому в данный проект введен элемент искусственного интеллекта для первичной аналитики отходов на точке сбора.

#### 1.4 Требования к проекту

1. Устранение недостатков: обязательное наличие системы обратной связи и добавление технологии сортировки батареек по типам.
2. Сохранение достоинств: сохранение базовой функции надежного накопления отходов, стремление к разумной себестоимости, возможность работы от сети.

## **2 КОНСТРУКТОРСКО-ТЕХНОЛОГИЧЕСКИЙ ЭТАП**

### **2.1 Методы проектирования при разработке конструкции**

Применялся функционально-стоимостной анализ (ФСА) для выбора оптимальных компонентов, обеспечивающих надежность при минимальной стоимости. Использован модульный принцип проектирования, что позволило разделить устройство на независимые блоки (электронный модуль, механизм приема, корпус) для упрощения сборки, тестирования и возможного ремонта.

### **2.2 Методы дизайнерской работы**

Для создания образа изделия применялся метод дизайн-мышления с упором на удобство для пользователя (UX). Были созданы эскизы и прототипы корпуса, продумано расположение дисплея и индикаторов для максимального удобства и простоты использования. Современный внешний вид, чтобы гармонично вписываться в различные интерьеры (от школьного класса до офиса).

### **2.3 Выбор материалов**

При выборе материалов учитывались технологичность, экологичность, прочность и эстетика. Для корпуса выбрана фанера (обработка на лазерном станке) как прочный, легкий для ремонта и экологичный материал. Для деталей механизма — PLA-пластик для 3D-печати, позволяющий создавать сложные формы с высокой точностью.

### **2.4 Использование АРИЗ**

Методы АРИЗ были применены для разрешения технического противоречия: необходимо было максимально точно идентифицировать тип батарейки, но не усложнять интерфейс для пользователя и не увеличивать время приема. Решением стала бесконтактная идентификация с помощью камеры и нейросети, которая происходит мгновенно и не требует от пользователя никаких действий.

## 2.5 Технологические карты

Изделие состоит из деталей, изготовленных из пластика и фанеры. Для оптимизации планирования и производства процесс описан в двух отдельных технологических картах (рисунок 1 и рисунок 2).

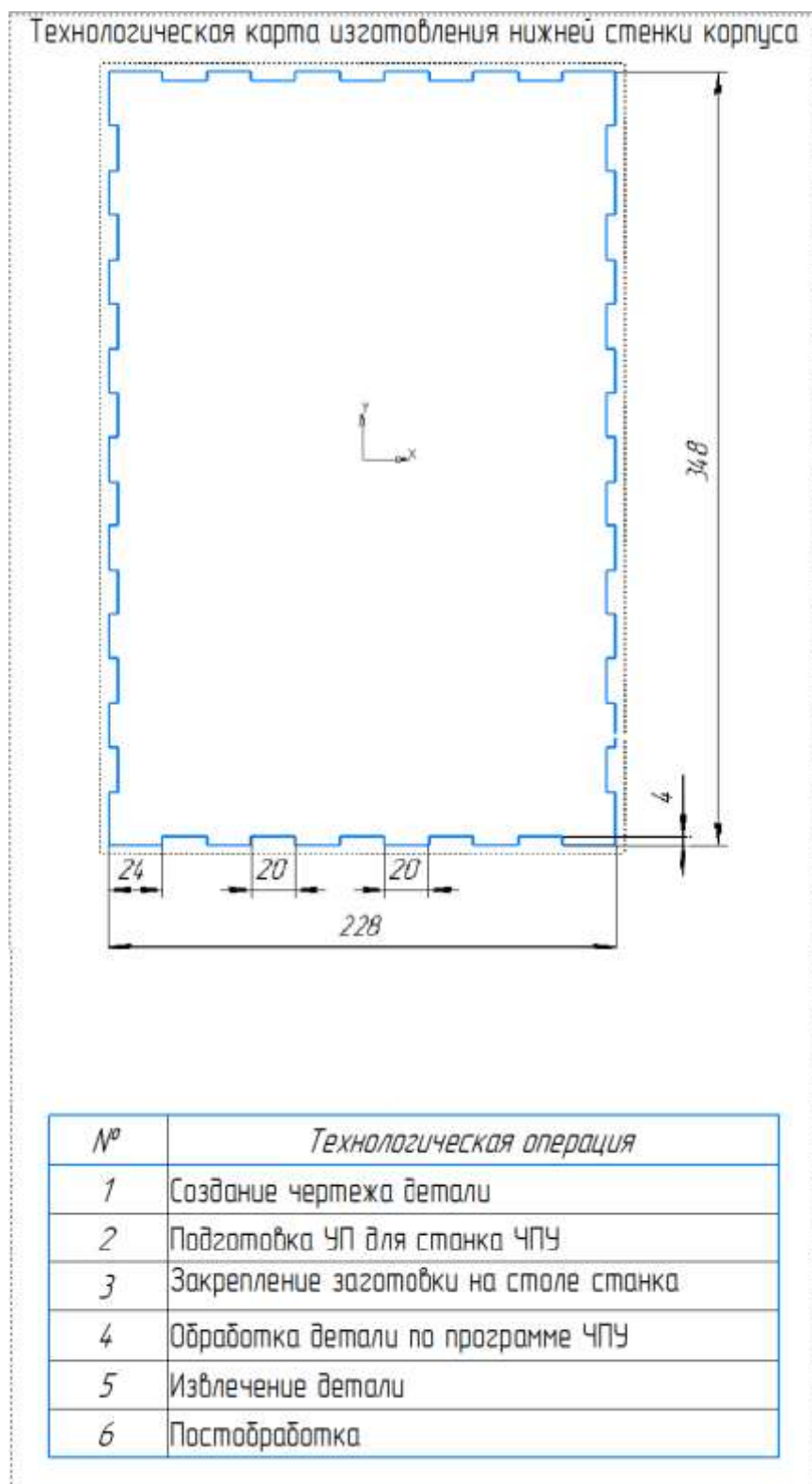
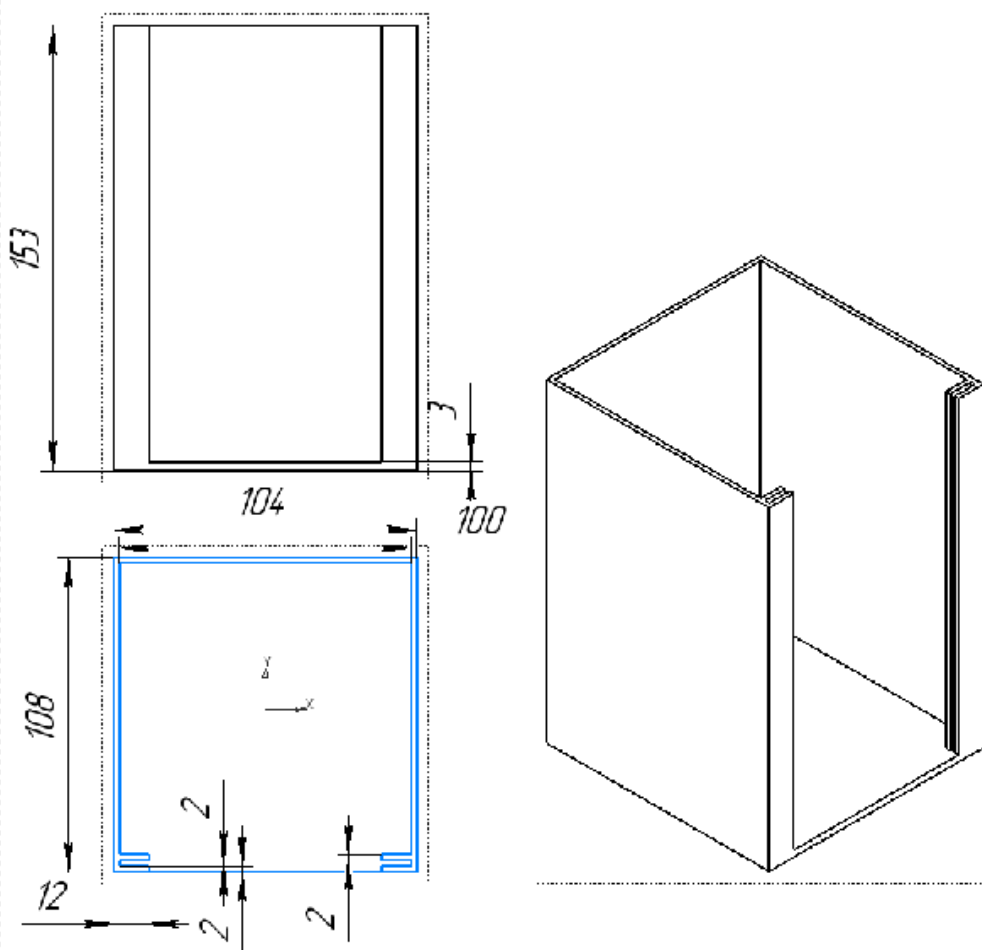


Рисунок 1 Технологическая карта изготовления нижней стенки корпуса

## Технологическая карта изготовления контейнера для хранения батареек



| № | Технологическая операция        |
|---|---------------------------------|
| 1 | Создание 3D модели              |
| 2 | Размещение модели в слайсере    |
| 3 | Создание файла формата GCODE    |
| 4 | Подготовка 3D принтера к печати |
| 5 | Печать модели на 3D принтере    |
| 6 | Извлечение детали               |
| 7 | Постобработка                   |
| 8 | Вырезание окошка из оргстекла   |
| 9 | Вставка окошка в контейнер      |

Рисунок 2 Технологическая карта изготовления контейнера для хранения батареек

## 2.6 Выбор инструментов, оборудования, приспособлений и технологии изготовления

Выбор лазерного станка для раскроя корпуса обоснован высокой точностью, чистотой краев и возможностью быстрого тиражирования. Выбор 3D-печати для деталей механизма обоснован возможностью изготовления сложных по форме деталей (крепления, приемный лоток) без необходимости изготавливать специальные формы. Использование ручного инструмента (паяльник, набор отвёрток, плоскогубцы, кусачки) обосновано тем, что при создании прототипа нет необходимости в сложном промышленном оборудовании.

Таблица 2 Выбранные компоненты

| Название компонента и его назначение в проекте                                                                                 | Внешний вид                                                                           |
|--------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------|
| Arduino Uno<br>Управление исполнительными механизмами и интерфейсом.                                                           |   |
| Raspberry Pi<br>Используется для обработки изображения с камеры и идентификации на нем типов батареек с помощью нейросети [6]. |  |
| Сервопривод MG996R<br>Используется для вращения конвейерной ленты.                                                             |  |
| Сервопривод SG90<br>Предназначен для поворота распределяющей платформы.                                                        |  |

|                                                                                                                                                               |                                                                                     |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------|
| <p style="text-align: center;">RFID-RC522</p> <p style="text-align: center;">Предназначен для идентификации<br/>класса/отдела офиса пользователя.</p>         |   |
| <p style="text-align: center;">Камера DEXP DQ4M3FA1</p> <p style="text-align: center;">Нужна для получения изображения для<br/>дальнейшего распознавания.</p> |  |

## 2.7 Описание принципа работы

Основной принцип работы устройства состоит в следующем: пользователь подносит брелок, инициализирующий его класс/отдел офиса. Контейнер запускается и готов к приёму батареек. Пользователь помещает батарейку в специальное отверстие, где она попадает на конвейерную ленту. В этот момент камера снимает изображение батарейки и отправляет его на Raspberry Pi, для распознавания типа батарейки с помощью нейросети. После определения типа батарейки, микрокомпьютер отправляет сигнал на Arduino. Сервопривод поворачивает распределяющую платформу к нужному контейнеру. Сервоприводы запускают конвейерную ленту, батарейка падает на распределяющую платформу и скатывается в контейнер. После этого на монитор выводится обновленная статистика о сданных каждым классом/отделом батарейках.

В рамках проекта была разработана оригинальная гибридная технология изготовления, комбинирующая субтрактивную (лазерная резка плоских деталей корпуса) и аддитивную (3D-печать объемных и крепежных деталей) обработку. Это позволило оптимизировать процесс, сократив время и затраты на создание прототипа.

В рамках проекта создан комплекс программных продуктов:

1. Нейросетевая модель, обученная на собственном датасете.
2. Программа на Python для микрокомпьютера Raspberry Pi, обеспечивающая захват изображения с камеры, его предобработку,

3 Прошивка для Arduino Uno, принимающая команды от Raspberry Pi, управляющая системой приема и распределение батареек.

Электрическая схема устройства представлена на рисунке 3.

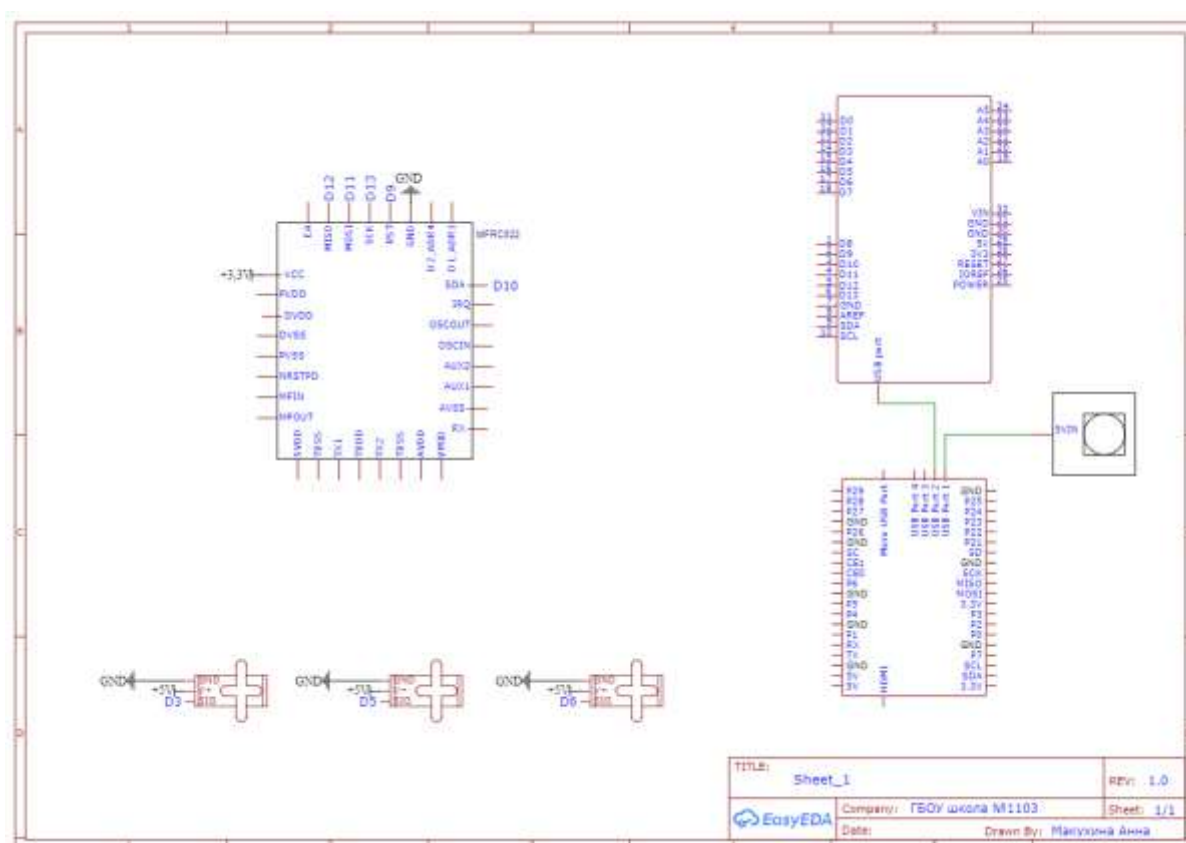


Рисунок 3 Электрическая схема

### **3 ЗАКЛЮЧИТЕЛЬНЫЙ ЭТАП**

#### **3.1 Экономическая оценка:**

- Arduino Uno – 678 руб.
- Raspberry Pi – 12 733 руб.
- Камера – 2 562 руб.
- Сервоприводы MG996R (2шт) – 588 руб.
- Сервопривод SG90 – 182 руб.
- RFID-RC522 – 141 руб.
- PLA-пластик – 285 руб.
- Фанера – 450 руб.
- Rubber-пластик – 301 руб.

Итоговая себестоимость прототипа – 17 920 руб. Розничная цена единичного прототипа может варьироваться в диапазоне от 25 000 до 30 000 руб., оптовая – от 22 000 до 28 000 руб.

При серийном производстве возможно удешевление себестоимости до 10 500 рублей. Тогда розничная цена будет составлять в районе 15 200 рублей, а оптовая в районе 13 500 рублей.

#### **3.2 Экологическая оценка**

- Материалы: основные материалы (фанера, PLA-пластик) являются экологичными и поддаются переработке. PLA — биоразлагаемый пластик.
- Технология изготовления: лазерная резка и 3D-печать — малоотходные технологии.
- Готовое изделие и эксплуатация: устройство способствует значительному снижению экологического ущерба за счет повышения сбора опасных отходов. Потребляет мало электроэнергии.
- Утилизация: Конструкция предусматривает легкую разборку для сортировки и переработки материалов. Электронные компоненты могут быть отправлены на специализированную утилизацию.

#### **3.3 Описание характеристик проектного изделия**

- 1 Обеспечивает сортировку, накопление и временное хранение батареек.



- 2 Реализует интерактивность (световая индикация, отображение статистики).
- 3 Осуществляет автоматический учет и визуализацию вклада пользователя.
- 4 Обладает современным эргономичным дизайном.

### **3.4 Уникальность проектного решения**

- 1 Оригинальная технология: разработана и применена гибридная технология изготовления, сочетающая лазерную резку (для деталей корпуса) и 3D-печать (для сложных объемных деталей и креплений).
- 2 Новый программный продукт: создано специализированное программное обеспечение для микроконтроллера и микрокомпьютера, способное распознавать батарейки и различать их по типу, реализующее логику работы устройства, ведение энергонезависимой статистики и взаимодействие с пользователем.
- 3 Оригинальное применение материалов: использование фанеры в качестве основного конструкционного материала для высокотехнологичного устройства является нестандартным решением, сочетающим экологичность, технологичность, прочность и эстетику.
- 4 Применение искусственного интеллекта для потребительского устройства: интеграция технологии машинного зрения и нейросетей в общедоступный контейнер для отходов является прорывным решением, переводящим сбор вторсырья на уровень данных и автоматизированной аналитики.

### **3.5 Перспектива тиражируемости**

Для перехода к серийному выпуску необходимо:

- Разработка специализированной печатной платы для повышения надежности.
- Создание технологической оснастки и конвейерной линии сборки.

### **3.6 Перспектива масштабируемости**

Проект может быть масштабирован за счет:

- Расширения функционала: адаптация концепции для сбора других видов вторсырья (пластиковые крышки, макулатура, мелкая

электроника) с соответствующей сменой информации, показывающей пользователю его вклад в экологию.

- Оснащение устройств IoT-модулями для создания городской сети «умных» эко-пунктов с удаленным мониторингом наполненности, сбором данных и подсчетом личного вклада пользователей и для передачи на сервер структурированных данных о типах и количестве собранных батареек, полученных нейросетью, для построения детальной статистики.

## **4 ЗАКЛЮЧЕНИЕ**

В результате выполнения проекта была решена комплексная конструкторско-технологическая задача «Разработка конструкции, изготовление и программирование функционального прототипа интерактивного контейнера для сбора и автоматической сортировки батареек с использованием нейронной сети».

### **4.1 Краткие выводы по результатам проекта**

Теоретическая и исследовательская работа завершена в полном объеме. Были проанализированы существующие аналоги, обоснована актуальность и социально-экологическая значимость проекта, выявлен спрос на мотивирующее решение и сформулированы технические требования. Конструкторско-технологический этап реализован успешно. Спроектирована оригинальная конструкция, сочетающая модульный корпус, изготовленный с применением гибридной технологии (лазерная резка фанеры и 3D-печать), и интеллектуальную систему управления. Разработана и обучена сверточная нейросеть на собственном датасете для классификации типов батареек, а также создано комплексное программное обеспечение для связки микрокомпьютера Raspberry Pi (обработка изображений, работа нейросети) и микроконтроллера Arduino Uno (управление механикой и интерфейсом).

Цель проекта достигнута: создан работоспособный прототип «умного» контейнера для сортировки батареек. Устройство не только накапливает

отходы, но и в реальном времени распознает их тип, обеспечивая пользователя интерактивной обратной связью и ведя энергонезависимый учет.

#### **4.2 Оценка полноты решений поставленных задач**

Все поставленные задачи, включая анализ, проектирование, разработку нейросетевой модели, создание ПО, изготовление и сборку, были решены. Полученный результат полностью соответствует первоначально сформулированным требованиям: устройство надежно, интерактивно, обладает уникальной функцией автоматической идентификации и имеет современный эргономичный дизайн.

#### **4.3 Рекомендации и исходные данные по конкретному использованию результатов проекта**

Устройство готово к пробной эксплуатации в школах, колледжах, эко-центрах и офисах. Для организации серийного производства необходимо провести работы по технологической подготовке: разработать специализированную печатную плату и создать необходимую технологическую оснастку.

#### **4.4 Результаты технико-экономической оценки внедрения проекта**

- 1 Себестоимость прототипа – 17 920 руб. При серийном производстве себестоимость снижается до 10 500 руб. за счет оптовых закупок и замены микрокомпьютера на Orange Pi.
- 2 Цены для рынка:
  - Розничная – 15 200 руб.
  - Оптовая – 13 500 руб. (при заказе от 100 шт.)
- 3 Эффективность:
  - Для производителя: рентабельность при опте ~45%. Окупаемость подготовки серийного производства — 12-18 месяцев.
  - Для заказчика: Инструмент для экопросвещения с измеримым результатом, улучшение репутации как современной и прогрессивной организации.

- Для государства/экологии: предотвращение ущерба природе (1 контейнер, собрав 500 батареек, спасает ~10 000 м<sup>2</sup> почвы).

#### **4.5 Результаты оценки научно-технического уровня выполненного проекта**

Научно-технический уровень проекта является высоким. В отличие от существующих аналогов, в нем впервые для бытовых контейнеров реализована технология машинного зрения на основе нейросети, что обеспечивает не только вовлечение пользователя, но и первичную аналитику состава отходов. Сочетание аддитивных и субтрактивных технологий изготовления, разработка оригинального ПО и применение методов АРИЗ для решения технических противоречий подтверждает комплексный инновационный характер работы. Проект соответствует современным тенденциям в области экологических технологий и социально-ориентированных технологических решений.

## 5 СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

- 1 Ассоциация содействия экономике замкнутого цикла «Ресурс» [Электронный ресурс]. – Режим доступа: <https://resurs2030.ru/> (дата обращения: 25.12.2025).
- 2 Всемирный день окружающей среды: как россияне проявляют свою заботу о природе [Электронный ресурс]. – Режим доступа: [https://www.insomar.ru/news/news-1\\_450.html](https://www.insomar.ru/news/news-1_450.html) (дата обращения: 25.12.2025).
- 3 «М.Видео»: 17% россиян сдают батарейки на утилизацию. [Электронный ресурс]. – Режим доступа: <https://asi.org.ru/news/2023/08/03/opros-17-rossiyan-sdayut-batareiki-na-utilizacziyu/> (дата обращения: 25.12.2025).
- 4 Экообменник. [Электронный ресурс]. – Режим доступа: <https://itecosolutions.ru/> (дата обращения: 28.12.2025).
- 5 Контейнеры для сбора батареек [Электронный ресурс]. – Режим доступа: <https://экоурна.рф/kontejnery-dlya-sbora-batareek/> (дата обращения: 28.12.2025).
- 6 Петин В. А. Микрокомпьютеры Raspberry Pi. Практическое руководство. — СПб.: БХВ-Петербург, 2015. - 240 с.: ил. - (Электроника).

ПРИЛОЖЕНИЕ

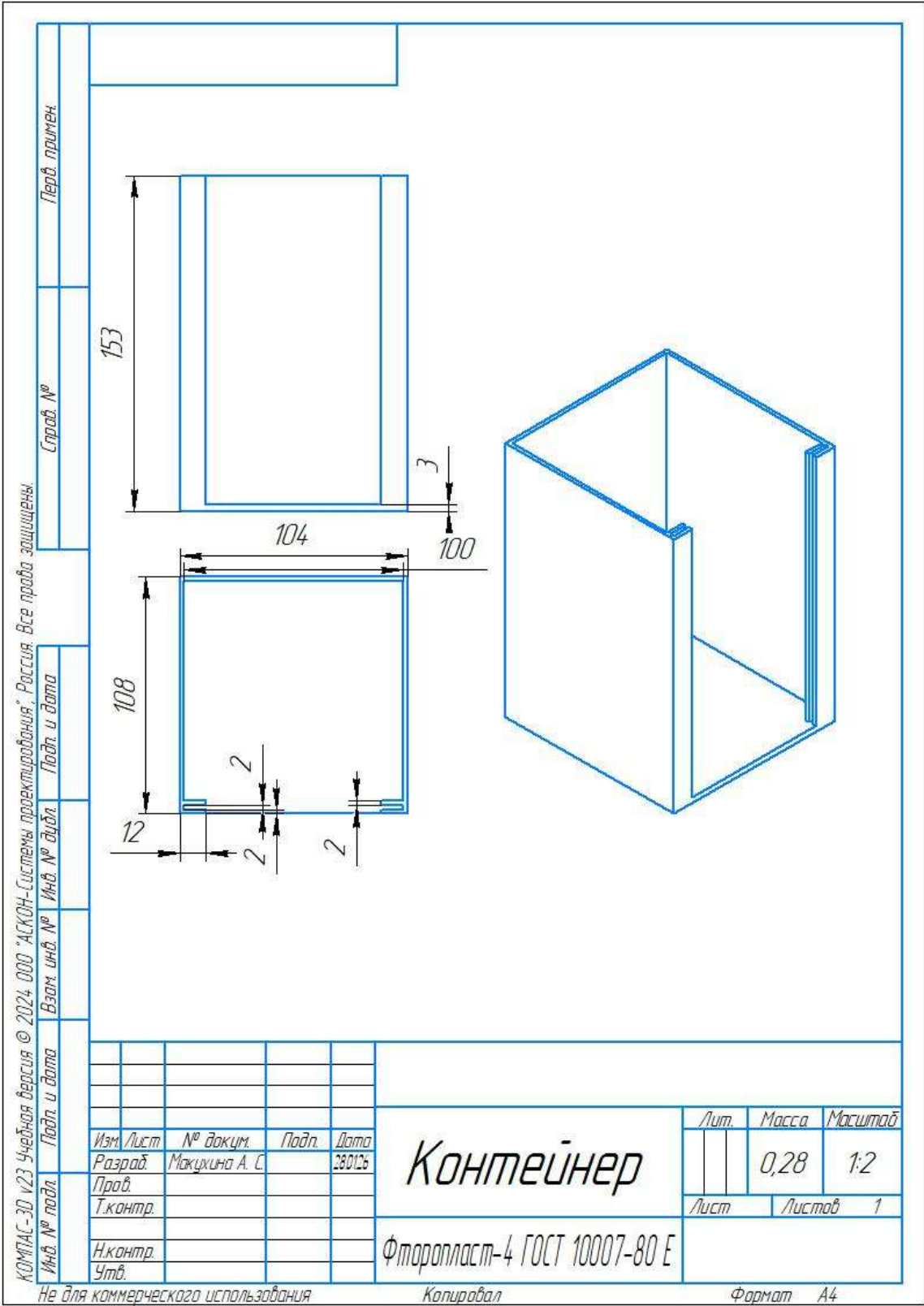


Рисунок 4 Чертеж контейнера для хранения батареек

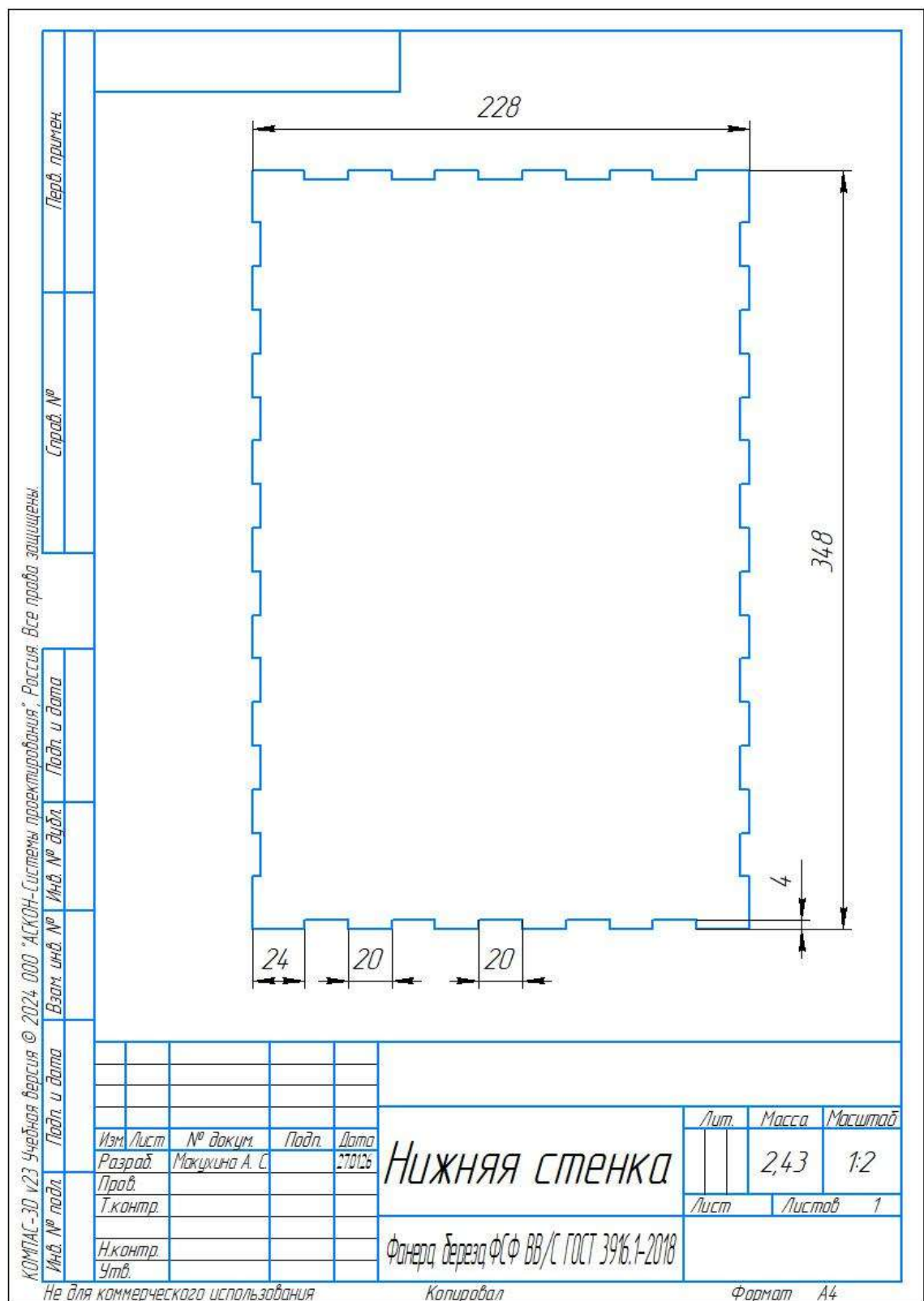
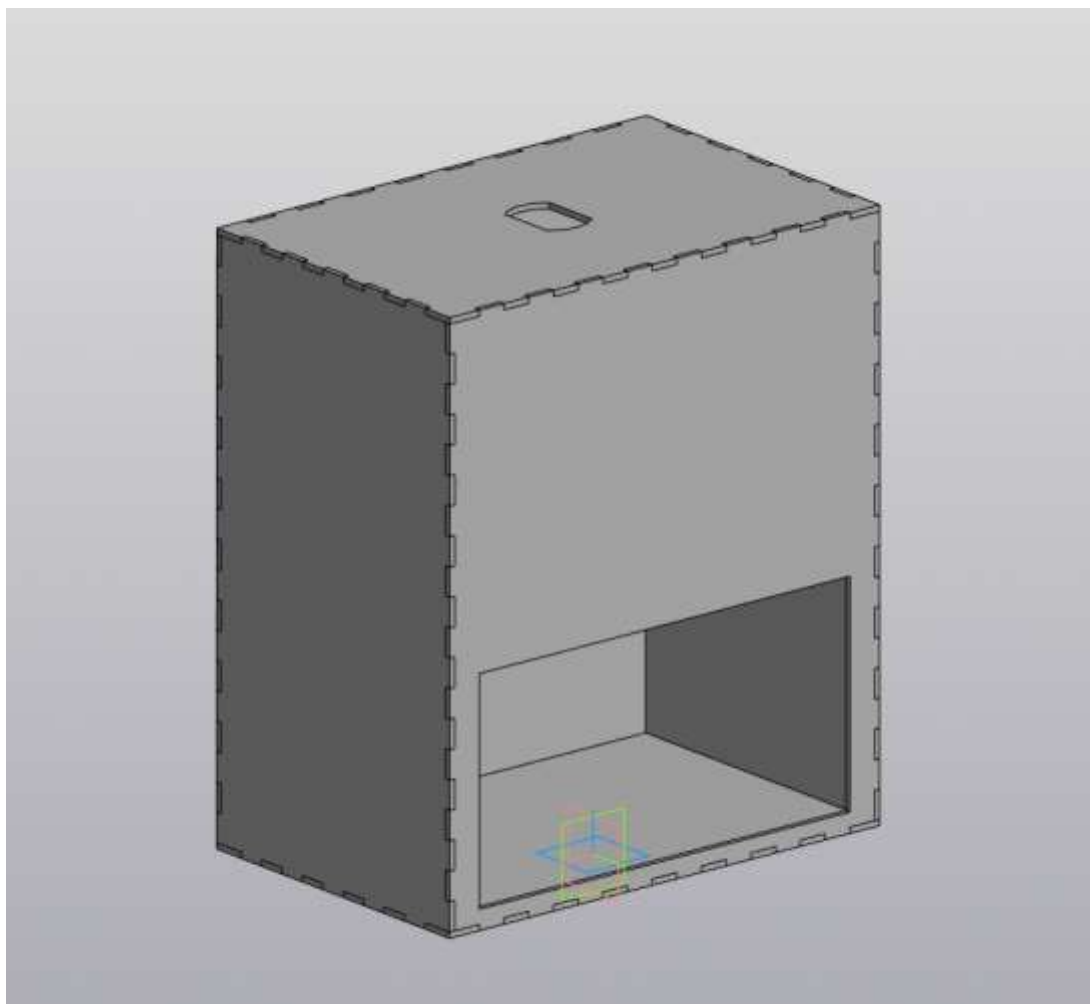


Рисунок 5 Чертеж нижней стенки корпуса



*Рисунок 6 Сборка корпуса*



## Программный код:

### Код для Arduino UNO

```
#include <SPI.h>
#include <MFRC522.h>
#include <Servo.h>

// ===== RFID =====
#define SS_PIN 10
#define RST_PIN 9
MFRC522 rfid(SS_PIN, RST_PIN);

// ===== SERVO PINS =====
#define SERVO_CONT_1_PIN 3
#define SERVO_CONT_2_PIN 5
#define SERVO_POS_PIN 6

// ===== DEFINES =====
#define SERVO_RUN_TIME 2000
// мс
#define COMMAND_TIMEOUT 5000 // мс — ожидание команды от Raspberry Pi

// ===== CONTINUOUS SERVO CALIBRATION =====
#define SERVO_STOP1 90
#define SERVO_STOP2 90
#define SERVO_RUN_1 110
#define SERVO_RUN_2 110

// ===== POSITION SERVO ANGLES =====
#define SERVO_POS_1 15
#define SERVO_POS_2 45
#define SERVO_POS_3 90

// ===== DEBUG =====
#define DEBUG_OUTPUT 1

// ===== STATE MACHINE =====
enum State {
  IDLE,
  WAIT_COMMAND,
  RUN_CONT_SERVOS,
  MOVE_POSITION_SERVO
};

State currentState = IDLE;
unsigned long stateStartTime = 0;

// ===== SERVO OBJECTS =====
Servo servo1;
Servo servo2;
Servo servo3;

// ===== SERVO 3 TARGET =====
int servo3TargetPos = SERVO_POS_1;

// ===== RFID UID → CLASS =====
String getClassFromUID(byte *uid, byte size) {
  if (size == 4 && uid[0] == 0xF3 && uid[1] == 0xB8 && uid[2] == 0x92 && uid[3] == 0xA5) {
    return "9I";
  }

  if (size == 4 && uid[0] == 0x11 && uid[1] == 0x22 && uid[2] == 0x33 && uid[3] == 0x44) {
    return "7B";
  }

  return "UNKNOWN";
}

void setup() {
  Serial.begin(9600);

  SPI.begin();
  rfid.PCD_Init();

  servo1.attach(SERVO_CONT_1_PIN);

  servo2.attach(SERVO_CONT_2_PIN);
  servo3.attach(SERVO_POS_PIN);

  servo1.write(SERVO_STOP1);
  servo2.write(SERVO_STOP2);
```

```

servo3.write(SERVO_POS_1);

#if (DEBUG_OUTPUT)
  Serial.println("READY");
#endif
}

void loop() {
  switch (currentState) {

    // ===== IDLE
    =====
    case IDLE:
      handleRFID();
      break;

    // ===== WAIT
    COMMAND
    =====
    case WAIT_COMMAND:
      handleCommand();
      break;

    // ===== RUN
    CONTINUOUS          SERVOS
    =====
    case RUN_CONT_SERVOS:
      if (millis() - stateStartTime >=
SERVO_RUN_TIME) {
        servo1.write(SERVO_STOP1);
        servo2.write(SERVO_STOP2);

        stateStartTime = millis();
        currentState =
MOVE_POSITION_SERVO;
      }
      break;

    // ===== MOVE
    POSITION          SERVO
    =====
    case MOVE_POSITION_SERVO:
      servo3.write(servo3TargetPos);
      #if (DEBUG_OUTPUT)
        Serial.print("SERVO3 MOVED
TO ");
        Serial.println(servo3TargetPos);
      #endif
      currentState = IDLE;
      break;
    }
  }

// ===== RFID HANDLER =====

```

```

void handleRFID() {
  if (!rfid.PICC_IsNewCardPresent())
return;
  if (!rfid.PICC_ReadCardSerial())
return;

  String      schoolClass      =
getClassFromUID(rfid.uid.uidByte,
rfid.uid.size);

  Serial.print("CLASS:");
  Serial.println(schoolClass);

  rfid.PICC_HaltA();
  rfid.PCD_StopCrypto1();

  stateStartTime = millis();
  currentState = WAIT_COMMAND;
}

// ===== SERIAL COMMAND
HANDLER =====
void handleCommand() {
  // таймаут ожидания команды
  if (millis() - stateStartTime >
COMMAND_TIMEOUT) {
    #if (DEBUG_OUTPUT)
      Serial.println("CMD TIMEOUT");
    #endif
    currentState = IDLE;
    return;
  }

  if (!Serial.available()) return;

  String      cmd      =
Serial.readStringUntil('\n');
  cmd.trim();

  // ===== RUN =====
  if (cmd == "RUN") {
    #if (DEBUG_OUTPUT)
      Serial.println("RUN      CONT
SERVOS");
    #endif
    servo1.write(SERVO_RUN_1);
    servo2.write(SERVO_RUN_2);

    stateStartTime = millis();
    currentState =
RUN_CONT_SERVOS;
  }
}

```

```

// ===== MOVE COMMANDS
=====
else if (cmd == "move1") {
    servo3TargetPos =
SERVO_POS_1;
#ifdef (DEBUG_OUTPUT)
    Serial.println("MOVE1
RECEIVED");
#endif
}
else if (cmd == "move2") {
    servo3TargetPos =
SERVO_POS_2;
#ifdef (DEBUG_OUTPUT)
    Serial.println("MOVE2
RECEIVED");
#endif
}
else if (cmd == "move3") {
    servo3TargetPos =
SERVO_POS_3;
#ifdef (DEBUG_OUTPUT)
    Serial.println("MOVE3
RECEIVED");
#endif
}

// ===== SKIP =====
else if (cmd == "SKIP") {
#ifdef (DEBUG_OUTPUT)
    Serial.println("SKIP");
#endif
    currentState = IDLE;
}
}

```

## Коды для Raspberry Pi

```
import threading
import time

from detector import BatteryDetector
from serial_worker import SerialWorker
from web import run_web

def main():
    detector =
    BatteryDetector(model_dir="tensorflow")
    serial_worker =
    SerialWorker("/dev/ttyUSB0", 9600)

    # Слушаем CLASS:*
    threading.Thread(
        target=serial_worker.listen_class,
        daemon=True
    ).start()

    # Веб
    threading.Thread(
        target=run_web,
        daemon=True
    ).start()

    print("🚀 Система запущена")

    while True:

        detector.classify_once_and_act(serial_worker)
        time.sleep(0.5)

if __name__ == "__main__":
    main()

# detector.py
import tensorflow.compat.v1 as tf
import numpy as np
from PIL import Image
import cv2
import time
import os
from stats import get_class, add_result

class BatteryDetector:
    """Детектор батареек и компонентов
    для камеры №1"""

    def __init__(self, model_dir='tensorflow'):
```

## Инициализация детектора

```
Args:
    model_dir: папка с моделью
TensorFlow
"""
self.model_dir = model_dir
self.camera_id = 1 # Камера №1
self.labels = self._load_labels()
self.session = None
self.input_tensor = None
self.scores_tensor = None
self.classes_tensor = None

self._setup_tensorflow()
self._load_model()

def _setup_tensorflow(self):
    """Настройка TensorFlow"""
    # Подавляем предупреждения
    tf.compat.v1.logging.set_verbosity(tf.compat
.v1.logging.ERROR)
    tf.disable_eager_execution()

    def _load_labels(self):
        """Загрузка меток классов"""
        labels_file = os.path.join(self.model_dir,
'labels.txt')
        if os.path.exists(labels_file):
            with open(labels_file, 'r',
encoding='utf-8') as f:
                labels = [line.strip() for line in f if
line.strip()]
            print(f"Загружены метки:
{labels}")
            return labels
        else:
            print("Файл labels.txt не найден,
используем стандартные метки")
            return ["aa", "crone", "not battary"]

    def _load_model(self):
        """Загрузка модели TensorFlow"""
        try:
            model_path =
os.path.join(self.model_dir,
'saved_model.pb')

            if not os.path.exists(model_path):
                raise FileNotFoundError(f"Файл
модели не найден: {model_path}")

            print("Загрузка модели...")
```

```

        # Загружаем граф модели
        with tf.gfile.GFile(model_path, 'rb')
as f:
    graph_def =
tf.GraphDef().FromString(f.read())

    graph = tf.Graph()
    with graph.as_default():
        tf.import_graph_def(graph_def,
name="")

    # Создаем сессию
    self.session =
tf.Session(graph=graph)

    # Получаем тензоры
    self.input_tensor =
graph.get_tensor_by_name('image_tensor:0')
    self.scores_tensor =
graph.get_tensor_by_name('detected_scores:
0')
    self.classes_tensor =
graph.get_tensor_by_name('detected_classes:
0')

    print("✓ Модель загружена")

except Exception as e:
    print(f'X Ошибка загрузки модели:
{e}')
    raise

def detect_frame(self, frame):
    """
    Детекция объектов на кадре

    Args:
        frame: numpy array изображения
        (BGR от OpenCV)

    Returns:
        Список обнаруженных объектов с
        confidence > 50%
    """
    try:
        # Конвертируем BGR в RGB
        rgb = cv2.cvtColor(frame,
cv2.COLOR_BGR2RGB)

        # Создаем PIL изображение и
        ресайзим

```

```

        img =
Image.fromarray(rgb).resize((320, 320))

        # Подготавливаем для модели (0-
        255, без нормализации!)
        img_array = np.array(img,
dtype=np.float32)
        img_array =
np.expand_dims(img_array, axis=0)

        # Запускаем детекцию
        scores, classes = self.session.run(
            [self.scores_tensor,
self.classes_tensor],
            feed_dict={self.input_tensor:
img_array}
        )

        # Обработка результатов
        if not hasattr(scores, '__len__') or
scores.shape == ():
            scores = np.array([scores])
            classes = np.array([classes])

        detected = []

        for i in range(len(scores)):
            confidence = float(scores[i])
            if confidence > 0.5: # порог 50%
                class_id = int(classes[i]) if i <
len(classes) else 0
                label = self.labels[class_id] if
class_id < len(self.labels) else
f'obj_{class_id}'

                detected.append({
                    'label': label,
                    'confidence': confidence,
                    'class_id': class_id
                })

        return detected

except Exception as e:
    print(f"Ошибка детекции: {e}")
    return []

def monitor_camera(self,
show_preview=False):
    """
    Мониторинг камеры №1

    Args:

```

```

        show_preview: показывать ли окно
с превью
        """
        print("=" * 50)
        print("ДЕТЕКТОР БАТАРЕЕК И
КОМПОНЕНТОВ")
        print("=" * 50)
        print(f'Камера: #{self.camera_id}')
        print("Поиск: 'aa' (батарейка) и 'crone'
(компонент)")
        print(f'Превью: {'ВКЛ' if
show_preview else 'ВЫКЛ'})")
        print("Нажмите Ctrl+C для
остановки")
        print("=" * 50)

        # Открываем камеру
        cap = cv2.VideoCapture(self.camera_id)
        if not cap.isOpened():
            print(f'X Ошибка: не удалось
открыть камеру {self.camera_id}')
            print("Проверьте подключение
камеры")
            return

        # Настраиваем камеру

        cap.set(cv2.CAP_PROP_FRAME_WIDTH,
640)

        cap.set(cv2.CAP_PROP_FRAME_HEIGHT,
480)

        last_check = 0
        check_interval = 1.0 # проверка
каждую секунду

        try:
            while True:
                # Читаем кадр
                ret, frame = cap.read()
                if not ret:
                    print("Ошибка чтения кадра")
                    time.sleep(0.1)
                    continue

                current_time = time.time()

                # Проверяем по времени
                if current_time - last_check >
check_interval:
                    # Детекция

```

```

                detected =
self.detect_frame(frame)
                last_check = current_time

                # Проверяем на aa и crone
                for obj in detected:
                    label = obj['label']
                    confidence = obj['confidence']

                    if label == 'aa':

                        print(f"[{time.strftime('%H:%M:%S')}] ⚠
ОБНАРУЖЕНА БАТАРЕЙКА 'aa!'
({confidence:.1%})")

                        if label == 'crone':

                            print(f"[{time.strftime('%H:%M:%S')}] ⚠
ОБНАРУЖЕН КОМПОНЕНТ 'crone!'
({confidence:.1%})")

                # Выводим все обнаруженные
объекты
                if detected:
                    print(f'Всего объектов:
{len(detected)}')
                    for obj in detected:
                        print(f" - {obj['label']}:
{obj['confidence']:.1%}")
                    print("-" * 40)

                # Показываем превью если
нужно
                if show_preview:
                    # Добавляем текст на превью
                    display = frame.copy()
                    cv2.putText(display, f'Camera
{self.camera_id} - Battery Detector',
(10, 30),
cv2.FONT_HERSHEY_SIMPLEX, 0.7, (0,
255, 0), 2)

                    # Статус
                    status_y = display.shape[0] - 20
                    status_text = "STATUS:
MONITORING"
                    cv2.putText(display, status_text,
(10, status_y),

                    cv2.FONT_HERSHEY_SIMPLEX, 0.6, (0,
255, 0), 2)

```

```

        cv2.imshow('Battery Detector',
display)

        # Выход по 'q' из окна превью
        if cv2.waitKey(1) & 0xFF ==
ord('q'):
            print("\nВыход по запросу
(кнопка 'q' в окне)")
            break

        except KeyboardInterrupt:
            print("\n\nОстановка по запросу
пользователя (Ctrl+C)")
        except Exception as e:
            print(f"\nОшибка: {e}")
        finally:
            # Очистка
            cap.release()
            if show_preview:
                cv2.destroyAllWindows()
            print("Камера закрыта")

    def single_check(self):
        """Однократная проверка камеры"""
        print("Однократная проверка
камеры...")

        cap = cv2.VideoCapture(self.camera_id)
        if not cap.isOpened():
            print(f"Ошибка: камера
{self.camera_id} недоступна")
            return

        try:
            # Захватываем кадр
            ret, frame = cap.read()
            if not ret:
                print("Не удалось получить
изображение")
                return

            # Детекция
            detected = self.detect_frame(frame)

            print("\n" + "="*40)
            print("РЕЗУЛЬТАТЫ
ПРОВЕРКИ:")
            print("="*40)

            if detected:
                aa_found = False
                crone_found = False

                for obj in detected:

```

```

                    label = obj['label']
                    confidence = obj['confidence']

                    print(f'- {label}:
{confidence:.1%}')

                    if label == 'aa':
                        aa_found = True
                        print(" ⚠ ВНИМАНИЕ:
Обнаружена батарея!")

                    if label == 'crone':
                        crone_found = True
                        print(" ⚠ ВНИМАНИЕ:
Обнаружен компонент!")

                    if aa_found and crone_found:
                        print("\n 🔴 КРИТИЧЕСКОЕ
ОБНАРУЖЕНИЕ: найдены и батарея и
компонент!")
                    elif aa_found:
                        print("\n 🟡 ОБНАРУЖЕНО:
батарея 'aa'")
                    elif crone_found:
                        print("\n 🟡 ОБНАРУЖЕНО:
компонент 'crone'")

                    else:
                        print("Объекты не обнаружены")
                        print(" 🟢 Безопасно")

                finally:
                    cap.release()

            def close(self):
                """Закрытие ресурсов"""
                if self.session:
                    self.session.close()
                    print("Ресурсы TensorFlow
освобождены")

            def classify_once_and_act(self,
serial_worker):
                """
                Однократное распознавание +
                команда Arduino
                """

                class_id = get_class()
                if not class_id:
                    return

                cap = cv2.VideoCapture(self.camera_id)
                ret, frame = cap.read()

```

```

cap.release()

if not ret:
    return

detected = self.detect_frame(frame)

result = "unknown"

for obj in detected:
    if obj["label"] == "aa":
        result = "aa"
        break
    elif obj["label"] == "crone":
        result = "crone"
        break

if result == "aa":
    serial_worker.send_move("move1")
elif result == "crone":
    serial_worker.send_move("move2")
else:
    serial_worker.send_move("move3")

serial_worker.run()
add_result(class_id, result)

print(f'✅ Класс {class_id}: {result}')

def main():
    """Основная функция"""
    try:
        # Создаем детектор
        detector =
BatteryDetector(model_dir='tensorflow')

        # Меню
        print("\nВыберите режим работы:")
        print("1 - Непрерывный мониторинг
(без превью)")
        print("2 - Непрерывный мониторинг
(с превью)")
        print("3 - Однократная проверка")
        print("0 - Выход")

        choice = input("\nВаш выбор: ").strip()

        if choice == '1':

detector.monitor_camera(show_preview=False)
elif choice == '2':

```

```

detector.monitor_camera(show_preview=True)
e)
        elif choice == '3':
            detector.single_check()
        elif choice == '0':
            print("Выход")
        else:
            print("Неверный выбор, запускаю
мониторинг без превью...")

detector.monitor_camera(show_preview=False)

except Exception as e:
    print(f"Ошибка: {e}")
finally:
    if 'detector' in locals():
        detector.close()

if __name__ == "__main__":
    # Проверяем OpenCV
    try:
        import cv2
        main()
    except ImportError:
        print("Ошибка: OpenCV не
установлен!")
        print("Установите: pip install opencv-
python")

```



```

import serial
import time
from stats import set_class

class SerialWorker:
    def __init__(self, port="/dev/ttyUSB0",
        baud=9600):
        self.ser = serial.Serial(port, baud,
            timeout=1)
        print("Serial открыт")

    def listen_class(self):
        """
        Слушает CLASS:9I от Arduino
        """
        while True:
            line =
self.ser.readline().decode(errors="ignore").strip()
            if line.startswith("CLASS:"):
                class_id = line.split(":", 1)[1]
                set_class(class_id)
                print(f"📦 Получен класс:
{class_id}")

            def send_move(self, move_cmd: str):
                print(f"Arduino: {move_cmd}")
                self.ser.write((move_cmd +
"\n").encode())

            def run(self):
                time.sleep(0.3)
                print("Arduino: RUN")
                self.ser.write(b"RUN\n")

import threading

lock = threading.Lock()

current_class = None

stats = {
    # "9I": {"aa": 0, "crone": 0, "unknown": 0}
}

def set_class(class_id: str):
    global current_class
    with lock:
        current_class = class_id
        if class_id not in stats:
            stats[class_id] = {"aa": 0, "crone": 0,
"unknown": 0}

```

```

def get_class():
    with lock:
        return current_class

def add_result(class_id: str, result: str):
    with lock:
        stats[class_id][result] += 1

def get_stats():
    with lock:
        return dict(stats)

from flask import Flask,
render_template_string
from stats import get_stats

app = Flask(__name__)

HTML = """
<!doctype html>
<html>
<head>
    <meta charset="utf-8">
    <title>Battery stats</title>
    <style>
        table { border-collapse: collapse; }
        td, th { border: 1px solid #444; padding:
6px 10px; }
    </style>
</head>
<body>
<h2>📊 Статистика батареек</h2>
<table>
<tr>
    <th>Класс</th>
    <th>AA</th>
    <th>Crone</th>
    <th>Unknown</th>
</tr>
{% for cls, v in stats.items() %}
<tr>
    <td>{{cls}}</td>
    <td>{{v.aa}}</td>
    <td>{{v.crone}}</td>
    <td>{{v.unknown}}</td>
</tr>
{% endfor %}
</table>
</body>
</html>
"""

```

```

@app.route("/")
def index():
    return render_template_string(HTML,
stats=get_stats())

def run_web():
    app.run(host="0.0.0.0", port=5000)

import threading
import time

from detector import BatteryDetector
from serial_worker import SerialWorker
from web import run_web

def main():
    detector =
BatteryDetector(model_dir="tensorflow")
    serial_worker =
SerialWorker("/dev/ttyUSB0", 9600)

    # Слушаем CLASS:*
    threading.Thread(
        target=serial_worker.listen_class,
        daemon=True
    ).start()

    # Веб
    threading.Thread(
        target=run_web,
        daemon=True
    ).start()

    print("🚀 Система запущена")

    while True:

detector.classify_once_and_act(serial_worke
r)
        time.sleep(0.5)

if __name__ == "__main__":
    main()

```